

	Politechnika Bydgoska im. Jana i Jędrzeja Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki Nowoczesne Technologie Przetwarzania Danych Laboratorium 10 Temat: Analiza danych w Spark SQL	
---	--	---

Sprawozdanie, Marcin Gregrowicz, semestr VI, grupa nr 1

Cel ćwiczenia

Celem ćwiczenia jest zapoznanie się z Spark SQL; wykonywanie zapytań SQL bezpośrednio na DataFrame lub na plikach zapisanych w formacie Parquet, CSV itd. obsługiwanych przez Spark.

Link do repozytorium Git:

<https://github.com/MarGegr/NTPD10>

Zadanie 1: Przygotowanie środowiska i podstawowe władowanie pliku Parquet

Po skonfigurowaniu w projekcie Apache Spark, wygenerowałem plik z danymi w postaci pliku Parquet. Do tego wykorzystany został plik `generate_data_parquet.py`, który stworzył dane reprezentujące słownik pracowników i zapisał je do folderu `pracownicy.parquet`.

Kod tego zadania znajduje się w pliku `Zad01.py`.

W zadaniu wyświetliłem pierwsze 5 wierszy, a także strukturę danych.

Zadanie 2: Ładowanie danych CSV i rejestracja jako widok tabelaryczny

Do tego zadania wygenerowałem plik z danymi w postaci pliku CSV. Do tego wykorzystany został plik `generate_data_csv.py`, który stworzył dane dotyczące transakcji i zapisał je do pliku `data.csv`.

Kod tego zadania znajduje się w pliku `Zad02.py`.

Wykonałem tutaj kwerendę `SELECT * FROM widok_transakcje LIMIT 10`, co zwróciło 10 pierwszych rekordów mojej tablicy.

Zadanie 3: Spark SQL. Tworzenie zapytań

Kod tego zadania znajduje się w pliku `Zad03.py`.

W tym zadaniu skorzystałem z danych wygenerowanych do zadań 1 i 2 (dotyczące pracowników i transakcji), dzięki temu mogłem potem połączyć dwa osobne widoki w zapytaniu SQL.

Wykonałem zapytanie SQL agregacje (`COUNT`, `SUM` i `AVG`), grupowanie (`GROUP BY` p.dział, t.region), warunkowe filtrowanie (`WHERE t.kwota_transakcji > 300.0`) oraz `JOIN` i wyświetliłem wyniki.

Stworzyłem też dodatkową kwerendę sprawdzającą działanie `MIN` i `MAX`.

Następnie przeanalizowałem je wyświetlając metryki statystyczne zbioru wynikowego korzystając z `describe()`.

Wykorzystałem również funkcję `explain()`. Wyświetla ona wewnętrzny plan wykonania zapytania wygenerowany przez optymalizator Sparka. Pokazuje ona krok po kroku, jak Spark zamierza przetworzyć dane zanim faktycznie zacznie to robić. Sprawdza składnię, semantykę oraz optymalizuje zapytanie (np. przesuwa filtrowanie `WHERE` przed `JOIN`, żeby drastycznie ograniczyć liczbę przetwarzanych wierszy).

Wynik tego zapytania na końcu zapisałem też do formatu CSV, znajduje się w pliku `raport_koncowy.csv`.